

ARIF 設計書

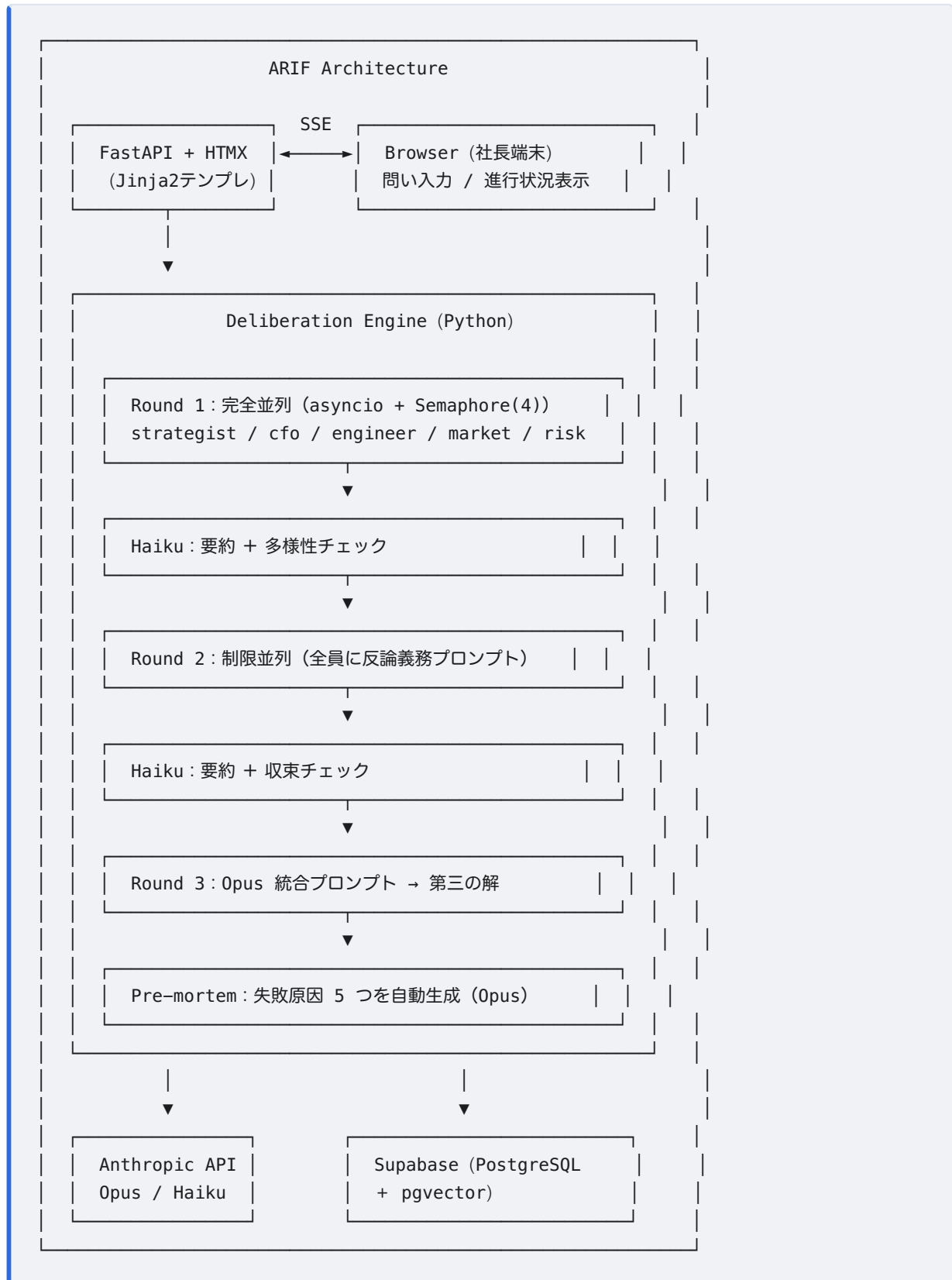
バージョン： 0.2

最終更新： 2026-05-26

ステータス： 設計諮問完了・上様確認待ち

1. アーキテクチャ概要

1.1 全体構成図



1.2 設計方針

方針	内容
フロントエンド	FastAPI + HTMX (Phase 3 まで) 。ユーザーは社長 1 名のため SPA は過剰。
非同期処理	<code>asyncio</code> + <code>AsyncAnthropic</code> で Round 1・2 を並列化
コスト制御	Round 間要約を Haiku に委託し、Opus の入力トークンを削減
堅牢性	1 体の失敗で全体停止させない設計 (エラー体はスキップして進行)
バージョン管理	モデルバージョンを <code>config.yaml</code> で一元ピン留め

2. モジュール構成

```
arif/
├── engine/
│   ├── agents.py      # エージェント定義・ペルソナ管理
│   ├── debate.py      # 3ラウンド熟議ループ (メインエンジン)
│   ├── memory.py      # Supabase 読み書き・RAG文脈生成
│   └── quality.py     # 多様性チェック・Pre-mortem収束判定
├── api/
│   ├── main.py        # FastAPI エントリポイント・SSE
│   ├── routes/
│   │   ├── deliberations.py  # /deliberations エンドポイント
│   │   └── context.py        # /context エンドポイント
│   └── templates/          # Jinja2 + HTMX テンプレート
├── config.yaml           # モデルバージョン・定数の一元管理
├── tests/
│   ├── test_debate.py
│   ├── test_memory.py
│   └── test_quality.py
└── docs/
    ├── 仕様書.md
    └── 設計書.md (本文書)
```

3. 設定ファイル (config.yaml)

```
models:
  deliberation: "claude-opus-4-20250514"      # 討論・統合・Pre-mortem
  summarization: "claude-haiku-4-20250514"    # 要約・収束チェック・補助
  embedding:    "text-embedding-3-small"      # pgvector 用ベクトル化

debate:
  concurrency_limit: 4                        # 同時API呼び出し上限
  max_tokens_per_agent: 2048                 # エージェント1体あたりの最大出力トークン
  summary_max_tokens: 1500                   # Haiku 要約の最大トークン
  synthesis_max_tokens: 2000                 # Round 3 統合の最大トークン
  timeout_seconds: 120                       # 1エージェント呼び出しのタイムアウト
  max_api_calls_per_session: 25              # セッションあたりのAPI呼び出し上限 (コスト暴走防止)

cost:
  target_usd_per_session: 0.20               # 1回の熟議コスト上限 (ドル)
  alert_threshold: 0.15                      # この金額を超えたらログに警告
```

4. エージェント設計 (agents.py)

4.1 データ構造

```
@dataclass
class Agent:
    id: str
    role: str
    system_prompt: str
    model: str
    temperature: float
    max_tokens: int = 2048
    optional: bool = False
```

4.2 温度パラメータの設計意図

エージェント	温度	意図
strategist	0.7	やや創造的。長期シナリオを多様に生成
cfo	0.3	保守的・数字重視。ブレない定量軸
engineer	0.4	現実的。実現可能性に集中
market	0.8	発散的。顧客視点の意外な気づきを促す
risk	0.2	最も保守的。最悪ケースを確実に拾う
philosopher	1.0	最も自由。常識を壊す問いを立てる
synthesizer	0.5	バランス型。統合と創造性を両立

温度パラメータを意図的に分散させることで、モデルが同一学習データを持っていても出力の多様性を構造的に確保する（群集効果防止の第一手段）。

5. 熟議フロー設計 (debate.py)

5.1 フロー詳細

```
入力: question (課題文字列)
↓
[前処理] memory.py で会社コンテキスト・過去議論を取得 (RAG)
↓
[Round 1] asyncio.gather で全エージェントを並列呼び出し
           各エージェントは相互の意見を参照しない
           失敗した体はスキップ (return_exceptions=True)
↓
[要約①] Haiku で Round 1 全意見を 1,500 tokens 以内に要約
           多様性スコアを計算 (コサイン類似度の逆数)
           多様性 < 0.3 の場合はログに警告
↓
[Round 2] asyncio.gather で全エージェントを並列呼び出し
           入力: 要約された Round 1 意見 + 会社コンテキスト
           プロンプト: 反論義務テンプレートを全員に適用
↓
[要約②] Haiku で Round 2 全意見を要約
           収束チェック (全員の意見が一致していないか確認)
↓
[Round 3] Opus で統合プロンプトを1回呼び出し
           入力: 要約① + 要約② + 「第三の解」品質基準
           出力: 第三の解 (synthesis)
↓
[Pre-mortem] Opus で「この判断が失敗した場合の原因 5 つ」を生成
↓
[保存] Supabase に全ログ・コスト・レイテンシを永続化
↓
出力: DebateResult (synthesis + pre-mortem + 全ラウンドログ)
```

5.2 想定レイテンシ

ステップ	推定時間
RAG 文脈取得	~2秒
Round 1（並列）	~30秒
Haiku 要約①	~5秒
Round 2（並列）	~45秒（入力が長いいため）
Haiku 要約②	~5秒
Round 3 統合	~30秒
Pre-mortem	~20秒
合計	約 2分 20秒~3分

SSE によるリアルタイム進行表示が必須。待ち時間の体感を軽減する。

5.3 コスト試算

項目	トークン概算	コスト概算
Round 1（5体 × input ~3,000）	~15,000	~\$0.045
Haiku 要約①	~5,000	~\$0.002
Round 2（5体 × input ~8,000）	~40,000	~\$0.120
Haiku 要約②	~3,000	~\$0.001
Round 3 統合	~10,000	~\$0.030
Pre-mortem	~5,000	~\$0.015
合計	~78,000	~\$0.213

目標 \$0.20 とほぼ同等。Haiku の活用と要約によりコストを抑制。
philosopher エージェントを ON にした場合は +\$0.02~0.04 増加。

6. データベース設計 (Supabase)

6.1 スキーマ

```

-- pgvector 拡張を有効化
CREATE EXTENSION IF NOT EXISTS vector;

-- =====
-- 熟議セッション
-- =====
CREATE TABLE deliberation_sessions (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title             TEXT NOT NULL,
    question          TEXT NOT NULL,
    status            TEXT NOT NULL DEFAULT 'pending',
    -- pending / round1 / round2 / round3 / completed / failed
    config            JSONB DEFAULT '{}',

    synthesis         TEXT,
    pre_mortem        TEXT,
    executive_summary TEXT,

    total_input_tokens INTEGER DEFAULT 0,
    total_output_tokens INTEGER DEFAULT 0,
    total_cost_usd      DECIMAL(10,4) DEFAULT 0,
    duration_seconds    INTEGER,

    created_at        TIMESTAMPTZ DEFAULT NOW(),
    updated_at        TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- 各エージェントの発言ログ
-- =====
CREATE TABLE agent_responses (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id        UUID NOT NULL REFERENCES deliberation_sessions(id) ON DELETE
    CASCADE,

    round            INTEGER NOT NULL CHECK (round BETWEEN 1 AND 3),
    agent_id         TEXT NOT NULL,
    agent_role       TEXT NOT NULL,
    model_used       TEXT NOT NULL,
    temperature      DECIMAL(3,2),

    content          TEXT NOT NULL,

    -- Haiku で後処理した構造化データ
    key_points       JSONB DEFAULT '[]',
    stance           TEXT CHECK (stance IN ('support','oppose','neutral','conditional')),
    confidence       DECIMAL(3,2) CHECK (confidence BETWEEN 0 AND 1),

    -- コスト・パフォーマンス追跡
    input_tokens     INTEGER,
    output_tokens    INTEGER,

```

```

cost_usd          DECIMAL(10,6),
latency_ms        INTEGER,
error             TEXT,    -- 失敗した場合のエラーメッセージ

created_at        TIMESTAMPTZ DEFAULT NOW()
);

-- 1ラウンドに同一エージェントは1回のみ
CREATE UNIQUE INDEX idx_unique_response
  ON agent_responses(session_id, round, agent_id);

-- =====
-- 会社コンテキスト (RAG 用)
-- =====
CREATE TABLE company_context (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category        TEXT NOT NULL,
  -- strategy / financial / org / market / risk / history
  title           TEXT NOT NULL,
  content         TEXT NOT NULL,
  embedding        vector(1536),
  source          TEXT,
  valid_from      DATE,
  valid_until     DATE,    -- NULL なら無期限
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- セッション横断の洞察
-- =====
CREATE TABLE deliberation_insights (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id      UUID NOT NULL REFERENCES deliberation_sessions(id),
  insight_type    TEXT NOT NULL,
  -- decision / concern / pattern / principle
  content         TEXT NOT NULL,
  embedding        vector(1536),
  president_rating INTEGER CHECK (president_rating BETWEEN 1 AND 5),
  president_note   TEXT,
  is_active       BOOLEAN DEFAULT TRUE,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- 社長フィードバック
-- =====
CREATE TABLE session_feedback (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id      UUID NOT NULL REFERENCES deliberation_sessions(id),
  overall_rating  INTEGER CHECK (overall_rating BETWEEN 1 AND 5),
  usefulness      INTEGER CHECK (usefulness BETWEEN 1 AND 5),

```

```

    novelty          INTEGER CHECK (novelty BETWEEN 1 AND 5),
    best_agent       TEXT,
    worst_agent      TEXT,
    free_comment     TEXT,
    action_taken     TEXT,    -- 実際にどう意思決定したか（後日入力）
    created_at       TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- pgvector インデックス
-- =====
CREATE INDEX idx_context_embedding ON company_context
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

CREATE INDEX idx_insights_embedding ON deliberation_insights
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

6.2 テーブル関係図

```

deliberation_sessions
├──
├── agent_responses (1セッション = 最大 15 レコード: 5体 × 3ラウンド)
├── deliberation_insights (セッションから抽出した洞察)
├── session_feedback (社長の事後評価)
└──

company_context (セッションと独立。全セッションから参照)

```

7. API 設計

7.1 エンドポイント一覧

メソッド	パス	説明
GET	/health	ヘルスチェック
POST	/deliberations	熟議セッションを開始
GET	/deliberations	過去のセッション一覧
GET	/deliberations/{id}	セッション詳細（全ラウンドログ含む）
GET	/deliberations/{id}/stream	SSE による進行状況リアルタイム配信
POST	/deliberations/{id}/feedback	社長フィードバック登録
GET	/context	会社コンテキスト一覧
POST	/context	会社コンテキスト登録・更新

7.2 POST /deliberations

```
// Request
{
  "question": "当社の5年後の主力事業をどう描くべきか?",
  "title": "中期事業戦略2026",
  "include_philosopher": false
}

// Response (即時返却。熟議はバックグラウンド実行)
{
  "session_id": "uuid",
  "status": "pending",
  "stream_url": "/deliberations/{uuid}/stream"
}
```

7.3 SSE イベント形式

```
event: round_start
data: {"round": 1, "message": "Round 1 開始"}

event: agent_thinking
data: {"round": 1, "agent": "strategist", "message": "経営戦略家が発言中..." }

event: agent_done
data: {"round": 1, "agent": "strategist", "content": "...意見テキスト..." }

event: round_summary
data: {"round": 1, "diversity_score": 0.72}

event: synthesis_done
data: {"synthesis": "...第三の解...", "pre_mortem": "...失敗原因..." }

event: complete
data: {"session_id": "uuid", "cost_usd": 0.187, "duration_seconds": 142}
```

8. エラーハンドリング設計

エラーケース	対処
1体のAPI呼び出し失敗	スキップして他のエージェントで継続
Rate Limit (429)	指数バックオフリトライ (最大3回、待機 1s / 2s / 4s)
タイムアウト (120秒超)	そのエージェントをスキップ
コスト上限超過	セッションを中断し部分結果を返す
Supabase 接続失敗	セッション結果をメモリに保持し後続の保存をリトライ
全エージェント失敗	500エラーを返しセッションを failed 状態に更新

9. 品質管理設計 (quality.py)

9.1 多様性チェック

Round 1 終了後、全エージェントの意見のコサイン類似度を計算し、多様性スコアを記録する。

多様性スコア = $1.0 - (\text{全ペアのコサイン類似度の平均})$
スコア < 0.3 : ログに警告 (プロンプト見直しの目安)

9.2 収束チェック

Round 2 終了後、全エージェントが同一の結論に達していないかを Haiku で判定する。
収束していた場合は Round 3 のプロンプトに「全員一致は異常。構造的原因を分析せよ」を追加する。

9.3 Pre-mortem (事前検死)

Round 3 の統合後、以下のプロンプトで Opus を 1 回呼び出す。

以下の経営判断が1年後に大失敗していました。
[synthesis の内容]

失敗の原因として最もありえるものを5つ、具体的に列挙してください。

10. 変更履歴

バージョン	日付	内容
0.1	2026-05-26	骨格作成
0.2	2026-05-26	Opus顧問審査結果を反映。エージェント数 8→5 体。FastAPI+HTMX 採用。スキーマ拡充。Pre-mortem・多様性チェック追加。